Massive Scale Data Analytics using Spark

Interim Report

Arbaaz Jaffer

BSc Computer Science

Supervised By: Dr. Daniel O'Keeffe

Royal Holloway, University of London

Date: December 19, 2025

Contents

1 Abstract **2**

2 Introduction **3**
2.1 The Problem . . . 3
2.2 Aims and Goals of the project . . . 3
2.3 Planning and time-scale . . . 4

3 Background Theory **5**
3.1 Big Data Characteristics and Challenges . . . 5
3.2 Distributed Computing Models . . . 5
3.2.1 MapReduce . . . 5
3.2.2 HDFS and Cluster Architecture . . . 6
3.3 Apache Spark Overview . . . 6
3.3.1 Spark Architecture . . . 6
3.3.2 RDDs, DataFrames, and Datasets . . . 7
3.3.3 Transformations, Actions, and Lazy Evaluation . . . 7
3.4 Performance Concepts . . . 8
3.4.1 Parallelism, Partitions, Shuffles . . . 8
3.4.2 Memory Management . . . 9
3.4.3 Spark Submit Parameters . . . 9

4 Implementation and Experimental Work **10**
4.1 Architecture of the Spark Application . . . 10
4.2 Data Source . . . 10
4.3 Pre-processing and Cleaning Pipeline . . . 10
4.4 Word Count Implementation (Problem 1) . . . 11
4.5 Distributed Grep Implementation (Problem 2) . . . 12
4.6 Experimental Design and Execution . . . 12
4.6.1 Small-scale Tests . . . 12
4.6.2 10 GB to 90 GB Tests . . . 12
4.6.3 Full 100 GB Tests . . . 13

5 Performance Analysis **14**
5.1 Experimental Setup . . . 14
5.2 Spark-submit Parameter Tuning . . . 14
5.2.1 Memory Settings . . . 14
5.2.2 Cores and Executors . . . 15
5.3 Scalability Benchmarks . . . 16
5.4 Interim Recommendations . . . 17

6 Software Engineering **18**
6.1 Methodology . . . 18
6.2 System Design . . . 18

1 Abstract

The rapid growth of data volume and complexity has raised the need to understand what exactly big data is, how to process it efficiently, and the challenges that come along with it. This also brings about the need for distributed computing frameworks and clusters of machines capable of handling such data as conventional systems using single computers would be obsolete in this context due to the sheer scale of data involved.

This leads to the key tool used within the project, Apache Spark, a distributed computing framework that has allowed for large scale data processing across clusters of computers in a fault-tolerant and extremely efficient manner. Some of the key features of Spark processes include in-memory computation, which drastically reduces the time it takes to process data compared to conventional methods. Additionally, Apache Spark supports the ability to process data via batch processing, which is useful for large datasets, as well as stream processing, which allows for real-time data analytics.

For this project, the aim is to explore the capabilities of Apache Spark, using the PySpark library to implement two programs that can handle large scale datasets. The first program will be a word count application that counts the frequency of each word in a large text file that would need to be cleaned and preprocessed before the word count can be performed. The second program will be a distributed grep application that searches for a key word or pattern in the same large text-based dataset and returns all titles containing that keyword or pattern.

Performance analysis will be conducted to evaluate the efficiency and scalability of the programs within this project by running them on the university's cluster and tuning various Spark-submit parameters such as executor memory, the number of cores and partitions of data. The results of these experiments will be displayed and discussed in detail, highlighting the strengths and weaknesses of the programs created, and providing an insight into the performance of Apache Spark for large scale data processing tasks. These results will also be used to provide recommendations for future workloads and optimisation strategies when using Apache Spark for big data analytics.

Overall, this project aims to provide a deeper understanding of PySpark and its capabilities for massive scale data processing, using clusters of computers in parallel to handle large quantities of data efficiently producing conclusions and recommended configuration settings.

2 Introduction

2.1 The Problem

Around 90 percent of the world's data has been generated in the last two years alone [1]. This rate of data generation is expected to continue growing exponentially, with estimates suggesting that over 4 years, the total amount of data created globally will continue to double [1]. This explosion of data has led to the emergence of the term "big data", which refers to these massive datasets that are too large and complex to be processed using traditional data processing methods [3]. The complexity and quantity of the data being generated will only continue to increase, making it essential to develop efficient methods for processing and analysing this data. Traditional data processing methods, which involve using a single machine to process the data, are no longer sufficient in the time taken and scalable enough to handle this increasing volume of data.

This has led to the development of distributed computing frameworks, such as Apache Hadoop and Apache Spark, which allow for the processing of large datasets across a cluster of machines, allowing for parallel processing and fault tolerance, making it possible to handle big data even if a single machine fails on the cluster. Apache Hadoop, which was one of the first distributed computing frameworks, is comprised of the Hadoop Distributed File System (HDFS) for storing data across multiple machines, MapReduce for processing data in parallel, and YARN for resource management. However, Apache Spark has emerged as a more efficient and faster alternative to Hadoop, due to its ability to perform in-memory computation, which drastically reduces the time taken to process data compared to Hadoop's disk-based approach [2]. This makes Spark more effective as it avoids repeated disk I/O cycles, allowing for faster processing for iterative or multi-stage computations.

2.2 Aims and Goals of the project

This project aims to use a large text-based dataset, specifically a 100 GB file comprised of Wikipedia articles, to implement two programs using PySpark, the Python API for Apache Spark. The first program is a word count application that counts the number of times each word within each page of the dataset appears. This is quite a standard benchmark to examine distributed processing behaviour and performance. The second program performs distributed grep, which searches for a specified word or pattern within the Wikipedia articles and returns the titles and all occurrences of that word or pattern. Both programs require the data to be cleaned and pre-processed before the main functionality can be implemented. Some of the challenges that arose during the implementation of dealing with such a large and complex dataset include cleaning and normalising the data, data partitioning and scalability across the cluster nodes.

A huge part of the project is the university cluster, where the Spark jobs were run across multiple nodes, allowing for the Wikipedia dataset to be processed in parallel across the cluster. This allowed for an evaluation of the performance and scalability of the programs by tuning the Spark-submit parameters, such as executor memory, number of cores and partitions of data. These experiments provided valuable insights into the performance of Apache spark for large scale text processing tasks, showing how a different number of resources and varying dataset sizes can affect the runtime of the implementation.

2.3 Planning and time-scale

This is the timeline I followed through the course of the project:

- **Week 1:** Initial project setup phase began and started to learn Apache Spark via the PySpark library. Reviewed some core Spark concepts and identified the Wikipedia XML dump as the main dataset to use.
- **Week 2:** No access to cluster so began local prototyping using a small CSV dataset to confirm PySpark installed and could practise using DataFrame operations.
- **Week 3 - 4:** Implemented the Wikipedia XML reading via a specialised package. Then filtered relevant pages to text based operations and ran a cleaning regex pipeline. Validated the clean outputs using small Wikipedia subsets.
- **Week 5 - 6:** Implemented the Word Count program using the DataFrame API. Verified correctness on a 1 GB subset, the proceeded to up-scale and use the larger datasets I stored in HDFS once given access to the Big Data Cluster.
- **Week 7 - 8:** The Distributed Grep program was implemented using the same cleaning pipeline, then applying a word matching function over the cleaned text data. Implemented this over the larger dataset size.
- **Week 9 - 10:** Designed and executed a structured performance analysis plan for the programs on the Big Data cluster. Kept a track of the runtimes of multiple different configurations used with `time spark-submit` and a variety of dataset sizes.
- **Week 11:** Data gathered on the excel sheet translated into plots using Matplotlib and pandas library.

3 Background Theory

3.1 Big Data Characteristics and Challenges

This project's key goal is understanding how to harness and process the knowledge that is hidden within the large, complex collections of information. Complicated and massive datasets that traditional data management systems cannot handle are referred to as big data [4]. A 100 GB XML Wikipedia dataset is used for the early deliverables of this project, which is perfect to showcase the power of distributed processing using a combination of Apache Spark and the university's cluster. Clearly, this massive dataset would prove difficult to handle without a distributed approach.

Some challenges that come with dealing with massive datasets can be cleaning the information. This can be broken into segments that can become very involved due to the quantity of data being processed. Data cleaning systems must be put in place due to the rising importance of having quality data [5] to perform experiments to analyse what the collection of data is indicating. Unclean data can be very costly due to the amount of processing power, resources and time that would need to be invested to end up with valuable data that can be used for further analysis [5].

3.2 Distributed Computing Models

3.2.1 MapReduce

Map and Reduce are key concepts to understand when delving into how distributed computation functions. These concepts were popularised by an open-source implementation of the MapReduce programming paradigm, called Apache Hadoop. The Map function has the role of taking raw input data, then processes it to generate a set of intermediate key-value pairs [6]. The Reduce function then takes the output of the Map function and merges the values that share the same key [6]. This step aggregates the data to produce a final smaller output, and when this is put into effect over large tasks, they can be broken down into independent sub-tasks that run in parallel significantly reducing the time taken to process the dataset.

Although MapReduce was a great and effective way to process massive datasets, it became glaringly obvious to researches at UC Berkeley that MapReduce was becoming inefficient for iterative computing jobs [2]. This bought about the idea to create Spark in the aims to make a simpler and faster implementation [2].

Apache Spark builds upon the foundational concepts of distributed, parallel processing across a cluster of machines, and as seen in the project, runs over the Hadoop Distributed File System (HDFS). MapReduce writes intermediate results to disk after each Map and Reduce stage, which is quite slow due to disk I/O latency. Spark performs in-memory storage for intermediate computations that allow for a much faster rate of processing large quantities of data [2]. A basic concept within Spark called Directed Acyclic Graph (DAG) is an engine that helps cyclic data flow [7]. By using a DAG scheduler that builds a complete lineage of transformations before execution, this allows Spark to combine multiple operations into a single stage and have fewer shuffle operations [7]. Lastly, fault-tolerance is another key feature that Spark achieves via Resilient Distributed Datasets (RDDs), which track the sequence of operations so if a partition is lost Spark can easily recompute only the missing data using that sequence to avoid unnecessary instructions. Whereas, MapReduce ensure fault tolerance by writing intermediate data to disk and

using pings to notify the master node of whether or not the computation has succeeded [6], which is quite sluggish in comparison to the speed that Spark provides.

3.2.2 HDFS and Cluster Architecture

Hadoop distributed file system (HDFS) is a "platform for storing, managing and distributing data" [9] across the machines within a cluster. The main objective that HDFS has is to store those large quantities of data and to supply a high throughput of data within a system [9]. Typically, the partitions are split into block sizes of 128 MB which can be distributed in parallel to allow for parallel processing through the various DataNodes in the cluster. Another key feature possessed by HDFS is the replication storage model, which allows for the blocks of data to be replicated three times and stored in three different DataNodes [9], which provides security for fault tolerance if a DataNode was to fail or be taken out of the cluster for maintenance.

A cluster is made up of a multitude of nodes that work in parallel to complete computations. A master node, also known as the NameNode in HDFS, can be seen as the 'brain' of the cluster, with its primary responsibilities being job management, scheduling tasks and coordinating the activities of all worker nodes [6][8]. The worker/slave nodes, known as DataNodes in HDFS, contribute to the task by storing data via a distributed file system such as HDFS and executing the computations assigned by the master node [8], they then report back to the master node, which can aggregate the final results and inform the user that the job they submitted is has been completed.

3.3 Apache Spark Overview

3.3.1 Spark Architecture

Sparks operational architecture is similar in physical layout to Hadoop, in that it uses master-worker nodes but its highly optimised and more effective for iterative jobs. Spark's superiority for iterative jobs stems from its ability to retain data in-memory, and since many machine learning algorithms use a function many times on the same dataset [10], this feature has revolutionised data processing which is seen by Spark outperforming Hadoop by 10x [10]. To do the same job, Hadoop would need to read and write from disk multiple times, which significantly slows down the processing time [10].

At the core of Spark's great performance improvements are several different components like the driver, executors and the cluster manager. The driver is the central brain of a Spark application as it runs the main function and is responsible for maintaining the information about the Spark application. Primarily, the driver negotiates resources from the cluster manager and work scheduling tasks on the executors [2]. The Directed Acyclic Graph (DAG) scheduler is built and optimised by the driver, separating the dependencies between the various stages of the job and creating a roadmap for execution [11][2] by modelling the data flow and intelligently groups multiple transformations into a single processing unit called a stage. A stage is a set of tasks that are executed in parallel across the cluster [2].

The cluster manager is a vital component that is responsible for acquiring the computing power across the cluster and allocating them to various Spark jobs [2]. Spark currently supports four cluster mangers, which are the internal stand-alone cluster manager, Apache Hadoop YARN, Apache Mesos and Kubernetes [2]. Finally, executors within

Spark are run on each worker node and are tasked with processing the data and returning the results to the driver.

3.3.2 RDDs, DataFrames, and Datasets

The key abstraction within Spark is the Resilient Distributed Dataset (RDD), a foundational data structure which is a read-only collection of objects that are partitioned across the nodes of the cluster and can be used in parallel [10]. One of the benefits provided by RDDs include their immutable nature, which allows for systems to alleviate the burden of slow or faulty nodes within the cluster, as they can simply recompute lost partitions using the lineage of transformations [12]. This process would then be carried out by a healthy worker node, ensuring that the overall computation can continue without any significant delays.

DataFrames and Datasets are higher-level concepts built upon RDDs that provide a more user-friendly interface for working with structured data. While RDDs are quite flexible and can handle unstructured data [12], they lack inherent structure forcing Spark to treat them as generic objects. DataFrames are similar to tables in a relational database, using rows and columns, allowing for a more intuitive way to work with data's content. Datasets are a type-safe version of DataFrames, providing "compile-time type safety" [2] which helps catch errors during development.

3.3.3 Transformations, Actions, and Lazy Evaluation

The Spark execution model is primarily based on two operations: transformations and actions. Transformations are operations that reconstruct a Spark DataFrame into a new DataFrame without effecting the original data [2], which gives it the property of permanence in the new structure. For example, an normal operation such as select() on a DataFrame will not change the original DataFrame [2], but will display the new results of the operation as another DataFrame. A key feature that transformations possess is that they are lazy, meaning when you write a transformation Spark would not execute the operation immediately; it merely creates the record of steps in the Directed Acyclic Graph (DAG) and does not trigger any computation on the data.

There are two types of transformations that can occur: narrow and wide [2]. These are crucial to Spark's ability to efficiently parallelise the workload. Narrow transformations are operations where each input partition maps to exactly one output partition [2]. These transformations are highly efficient due to their ability to execute without requiring data movement across the cluster and can be performed on the worker nodes where the data resides. On the other hand, wider transformations are where a single input partition contributes to multiple output partitions [2], which requires Spark to move and exchange data across the network between different worker nodes, known as a Shuffle.

This is where actions come into play, actions are operations that instruct Spark to start the actual computation on the dataset from the recorded transformations [2]. The result of these computations can then be returned to the driver node or be directly parsed to a storage system like Hadoop's Distributed File System (HDFS). Through a function like show() being executed within the code, the plan laid out by the DAG would then be executed over the dataset.

As we know all transformations throughout Spark are evaluated lazily, meaning that the results of these computations are not processed instantly and that they are instead recorded as a lineage [2]. Lazy evaluation is the key mechanism within Spark that waits

for an action to occur before performing any computation. The benefit of this approach stems from the Catalyst Optimiser being able to perform global optimisation across the entire pipeline [8], which then allows Spark to look at the complete DAG of operations and determine the most efficient path to take. This can be done by Spark joining or pipelining operations within the code and assigning them to a specific stage.

3.4 Performance Concepts

3.4.1 Parallelism, Partitions, Shuffles

Parallelism is a key factor as to why Spark is so efficient when it comes to processing massive quantities of data. It is achieved by spreading the workload across a number of machines, executing multiple tasks simultaneously instead of having a single machine carry the workload by itself [14]. The result of this dispersed approach to processing the large dataset is a significant reduction in the time it takes to complete the job, as multiple nodes work together to handle different parts of the data concurrently [14].

Some core laws heavily related to the experiments I conduct are, Amdahl's law, which states that the speed-up achieved by a parallel system is limited by the time required for the portion of the workload that is sequential [15]. This suggests that the benefits of parallelisation are not consistently scaling and will eventually flatten out. The formula presented for Amdahl's law [16] is:

$$S(n) = \frac{n}{1 + (n-1)f}$$

This formula shows the rate of increase in performance, S(n), as the number of processors, n, is increased with f being the part of the job that is not parallelised. The main implication of this law is that it explains the development of diminishing returns as there will be a fixed amount of time taken by the sequential shuffle operations within the Spark job, resulting in the limitation of the job's runtime.

Another important law is Gustafson's law, which argues that as the amount of processors increases, the size of the problem that can be handled also increases [16]. This law is relevant as it suggests that with more resources available within the cluster, larger datasets can be processed more efficiently, which is a key aspect of why Spark is so effective for analysis of big datasets. The formula for Gustafson's law [16] is:

$$S(n) = n - \alpha(n-1)$$

This formula indicates that the speed-up, S(n), increases linearly with the number of processors, n, minus the fraction of the workload that inherently cannot be parallelised, represented by α. It emphasises that as more processors are added, the overall performance can improve significantly, provided that the non-parallelisable part remains minimal.

To effectively process the data in parallel, Spark must first divide the data into smaller logical chunks called partitions. Each of these partitions can then be processed independently and in parallel across the slave nodes within the cluster [2]. The number of partitions directly influences the level of parallelism that is possible, as more partitions allow for more tasks to be executed simultaneously. However, too many partitions can lead to a large overhead in managing these tasks [2], as each task also comes with its own setup and finishing operations.

As mentioned earlier in the report, a shuffle is a costly operation in which data exchange is required by wide transformations. They force data to be moved across the network to be aggregated, which acts as a sequential bottleneck in the otherwise parallel processing of Spark. This is due to the fact that all tasks must complete their shuffle operations before the next stage can begin [2]. As a result, they have a massive impact on the overall performance, making it crucial to lower the number of shuffles involved in a Spark job where possible.

3.4.2 Memory Management

Efficient memory management is a critical aspect of Spark's performance, as it directly affects the way in which data is processed and stored during the computation. Spark uses a unified memory management model that allocates memory for two main purposes: execution and storage [2]. Execution memory is a space reserved for the main computation, such as shuffles, joins and sorts. Storage memory is dedicated to caching and persistent storing of the partitions originating from the DataFrame to be kept within RAM.

3.4.3 Spark Submit Parameters

The spark-submit command is used in order to run the Spark application over the cluster, There are a variety of adjustable parameters used within this command to define the computational resources being allocated to the job. This will be the fundamental to be able to finding performance testing aspects to the programs within this project. Some of the resource allocation parameters include the number of executors, which dictates the number of executors to launch on the cluster's worker nodes. There is also the executor cores parameter, which specifies the number of CPU cores to be assigned to each executor. This is another factor that that affect the level of parallelism as increasing this parameter allows for tasks to run concurrently on an individual executor. Another component would be the executor memory, which assigns the amount of RAM allocated to each executor.

4 Implementation and Experimental Work

4.1 Architecture of the Spark Application

The two programs implemented in this project share a common pipeline structure due to their nature. Some of those aspects are the initiation of the Spark environment, reading the raw XML dataset, filtering and cleaning which is followed by the specific logical operations based on what the program was implemented to achieve. This structure seems quite a handful but is actually made very efficient due to Spark's Lazy evaluation, which ensures that all these tasks are pipelined into an optimised execution plan.

The physical architecture involves the Big Data cluster used through the course of the project, it consists of 4 main components: the HDFS storage system, the Spark driver, the YARN cluster manager and the worker nodes within the cluster. HDFS is used to store the massive Wikipedia dataset on a distributed system. The Spark driver is the component that runs the python script, building the execution plan and coordinates the Spark job. YARN is the cluster manager, which works behind the scenes to allocate the resources requested in the prompt. Finally, the worker nodes run the parallel tasks, performing the computations and returning the results to the Driver node for the final results.

The project primarily relies on the DataFrame API within PySpark, which is a high-level abstraction chosen for its benefit presented by Spark's Catalyst Optimiser. It constructs a highly efficient execution plan and due to the tungsten engine providing faster execution rather than manual RDD code.

4.2 Data Source

The main source of data used to test both applications was the Wikipedia XML dump (around 100 GB), which was sourced via wget into the Big Data cluster, uncompressed and finally stored within the Hadoop Distributed File System (HDFS). Experimental runs were conducted with subsets of this 100 GB dataset which range from 10 GB up to 90 GB to analyse performance scaling due to its significance in the Big Data space.

Since the data source I chose is XML, I found a package that I utilised within the Spark execution which allows the files to be read as Spark DataFrames. Therefore, the `com.databricks:spark-xml` library was used. Some key options also included were the rowTag option to be implement over each `<page>` tag to correctly segment each Wikipedia article.

As the dataset being used

4.3 Pre-processing and Cleaning Pipeline

The pre-processing stage was identical for both programs, designed to normalise text and remove as much Wikipedia-specific markup and noise, ensuring the most accurate tokenisation and pattern matching with each respective program. Some steps taken in order to gather the correct information within the large Wikipedia XML dataset were to initially filter the pages by removing anything that was non-article pages, the to remove any redirect links as well as to get rid of empty pages.

After this selection and filtration process was complete, the remaining data, consisting of pages of Wikipedia articles, could now be used to start the cleaning process in order to make them useful for the programs being built. The cleaning process began with a

case normalisation along with a multitude of regex alteration functions on the text data, with the intent to remove things like Wiki links, templates and HTML tags present in the uncleaned text field.

All the cleaning performed was accomplished using PySpark's SQL functions (such as `regexp_replace()`, `lower()`, etc.) over using User-Defined Functions (UDFs) as they provide better performance and optimisation over the PySpark application. A important reason why this occurs is due to Spark SQL functions being understood by the Catalyst Optimizer, responsible for those added benefits of performance.

4.4 Word Count Implementation (Problem 1)

The Word Count program performs a large-scale statistical analysis over the Wikipedia Dump to determine the frequency of every word across the entire dataset. These are the steps taken to create the Distributed Word Count:

1. **Load the XML data from HDFS:** the Wikipedia dump is read from the HDFS file path using the park XML data source treating each `<page>` element as a record.
2. **Filter valid articles and select:** filtering and selecting the title and text columns within the pages.
3. **Cleaning the text content by:**
    - Converting all text to lowercase
    - Removing wiki-markup, HTML tags and URLs using regex
    - Filter out any non-alphabet characters
4. **Tokenise the cleaned text:** the cleaned text is then split into an array of words.
5. **Explode each word into rows:** the array is transformed so that each word has its own row, allowing for distributed aggregation.
6. **Filter out any invalid or unnecessary words:** removing any words that are would not be necessary in the final word count.
7. **Aggregate counts per word:** each word is grouped and counted to compute a global frequency for each unique word. This process is a wide transformation and triggers the shuffle phase.
8. **Ranking the results:** the counts are sorted into descending order and the top 20 results are taken to be displayed in the output (as a way to verify the computation was completed).

This is a pipeline and flow of the program as the code runs on each executor, in reality this code is run multiple times during the Spark job in order to clean the large dataset as the partitions are cleaned and their results are then aggregated within the driver node.

4.5 Distributed Grep Implementation (Problem 2)

The Distributed Grep is a far more efficient program as it avoids the global shuffle required to count the words in the Distributed Word Count program. The program aims to find a keyword or patterns within the Wikipedia dump.

1. **Define the keyword:** the keyword is defined to later be used within the matching logic to extract each occurrence.
2. **Repeat steps 1 - 3 from the Word Count program:** the problem requires for the text data to be utilised in order to find the occurrences of the word within the large dataset, the loading of the dataset and cleaning processes on each page from the first program can be implemented again.
3. **Filter matching documents:** the dataset is then filtered so only the rows (article pages) with the matching word is kept.
4. **Select output fields:** the resulting output is minimised down to the word itself and the title page in which it was found.

Unlike the Word Count program, the Distributed Grep program is primarily a filtering workload; it does not have the same need to perform a large number of shuffle operations as there is no global aggregation that takes place. Cleaning the data remains consistent throughout the process as its essential to perform these implementations like Word Count and Distributed Grep on a set of data without any of the usual markup and metadata within Wikipedia pages as it can provide false positives during each of these processes.

4.6 Experimental Design and Execution

4.6.1 Small-scale Tests

The main reasoning behind starting with miniaturised datasets (that were also Wikipedia articles) was to ensure the pipeline generalised to the full dataset before being given access to the Big Data cluster on the universities servers. The debugging of code before performing any sort of performance analysis was crucial as it would be a lot more inefficient to conduct on the larger datasets.

Initially, these smaller tests were performed locally as a way to learn what spark was capable of. Then, they become useful to clean the data on the cluster as a way to validate that the cleaned data was suitable for either the Word Count or Distributed Grep operation. This reduced any risk of spending cluster time on the large-scale runs, which would take quite long to process the data of that scale.

4.6.2 10 GB to 90 GB Tests

This phase of the experiment was designed to establish how runtime changes as the dataset size increases while controlling the parameters of the cluster in order to evaluate the results. The independent variable with every configuration on the cluster was the dataset size (10 GB, 20 GB, up to 100 GB), while all other factors were kept fixed:

- **Code Consistency:** the same code, which included the cleaning pipeline and program logic were utilised across each subset and for the distributed grep in particular the key word "queen" was kept consistent.

- **Resource Configuration:** for a given experimental configuration the parameters being allocated across the cluster (executors, cores and memory) remained constant to isolate the impact that data volume had on performance.

These subsets were generated from the original Wikipedia dump (100 GB XML dataset) and stored as a individual paths in HDFS. Each of the run times were repeated to ensure the initial result was reliable and consistent.

4.6.3 Full 100 GB Tests

The final-scale execution protocol focused on running the end-to-end pipeline on the full 100 GB XML dataset stored in HDFS. This stage served as the ultimate test of the distributed cluster's along with the implementations created.

Jobs were submitted through the YARN cluster manager using a standardised `spark-submit` template. To ensure the operational stability during execution, long-running 100 GB jobs were executed within tmux sessions, and recorded the outputs produced after completion.

Monitoring was performed via the Spark UI generated during execution, specifically focusing on:

- **The Jobs Page:** to observe stage progression and the impact of the shuffle phase in the Word Count job in particular.
- **The Executor Page:** to monitor task distribution and memory usage across the worker nodes on the cluster.
- **The DAG Visualisation:** to confirm that the Catalyst Optimiser correctly pipelines the multiple regex transformations into an efficient stage.

5 Performance Analysis

5.1 Experimental Setup

Two PySpark applications - a word count and a distributed grep - were evaluated using subsets of a larger Wikipedia XML dataset ranging from 10 GB up to 90 GB and finally the full 100 GB dump. By investigating into the physical structure of the cluster being used, the CIM help desk informed that the experiments were conducted on a cluster of ten virtual machine executors hosted on a single physical server. This means all Spark tasks ultimately shared the same underlying hardware resources such as the CPUs, memory (RAM), disk and network bandwidth.

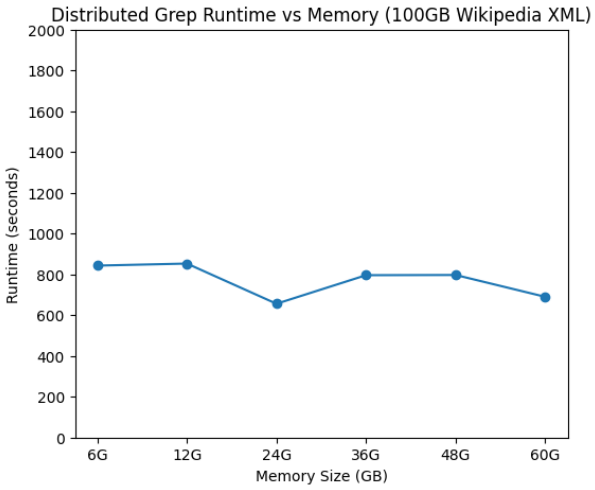
Each test was submitted with the Linux `time spark-submit` command to measure total job runtime, which was the primary metric recorded. The default HDFS block size (approx. 128 MB) was used for data partitioning. The focal point was on each jobs runtime under these controlled conditions.

5.2 Spark-submit Parameter Tuning

To gain an understanding of how each Spark configuration impacts performance, systematically varying each key parameter which includes the executor memory, CPU cores per executor and the amount of executors assigned to the Spark job. Within this section the effects of each will be presented and the test scenarios will be summarised.

5.2.1 Memory Settings

Each of the applications was run with different memory allocations to see if more memory speeds up processing. For the Distributed Grep job, the dataset was kept consistent as 100 GB, while using 6 executors with 4 cores each but varying the memory size allocated to each executor from 1 GB up to 10 GB. Similar test were carried out on the Word Count and it showed the same pattern although it took longer to process due to the nature of the shuffle operations occurring.



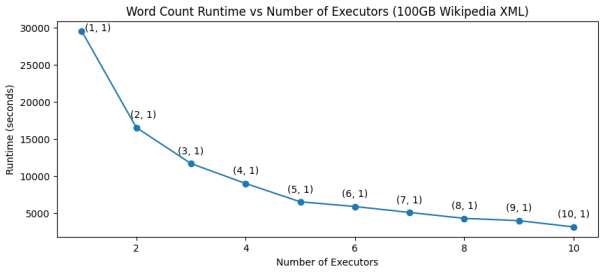
Overall, at the range of 4 GB - 6 GB per executor seemed to be the point where performance seemed to increase very slightly. This suggest that raising the amount of memory assigned to the jobs was not the limiting factor, which correlates to the

transformations taking places as the workload is CPU bound due to the computation being the focal point. In summary, since the memory when allocated at around 4 GB seems to suffice and adding more memory does not provide any significant increase in runtime, their was no performance boosts to allocate more than 4 GB. Since the Big Data server is used by many different users being responsible with the allocation of resources to a job that could take a significant amount of time to process is vital to the fair and considerate use of the cluster.

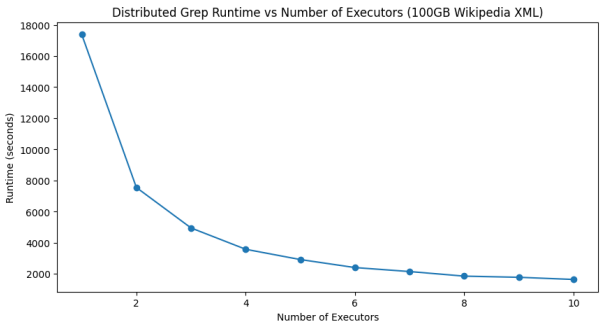
The minor fluctuations occurring can be put down to external factors like server load or the number of concurrent jobs which could cause small variations in the recorded run times.

5.2.2 Cores and Executors

For the Word Count program's performance tuning there were a series of experimental changes involving the executors and cores per executor made on the cluster, starting of with a linear growth of just the executors to discover how the increased parallelism affected the runtime.

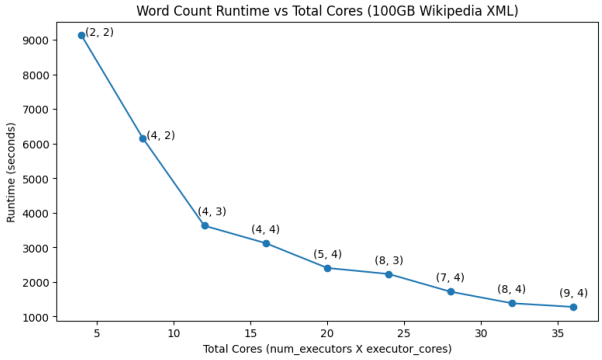


The graph displays a linear growth in the number of executors assigned to the Spark job and shows that there is a quite obvious initial performance gains via the sharp decrease in runtime when scaling from 1 to 4 executors. As more and more executors are assigned to the Spark job their impact on the runtime performance flattens significantly, which aligns with Amdahl's law, suggesting that speed-up in parallelised workloads will be increasingly limited by the sequential components of the job. In the Word Count program in particular, this is primarily a cause of the wide transformations required to group and count the words, which calls for a costly data shuffle to occur over the network. This results in a ceiling forming where the performance can no longer increase in a linear fashion and takes on an exponential form.



The same pattern reappears when the performance testing was conducted on the Distributed Grep program, showing a much steeper decline in runtime and then the same

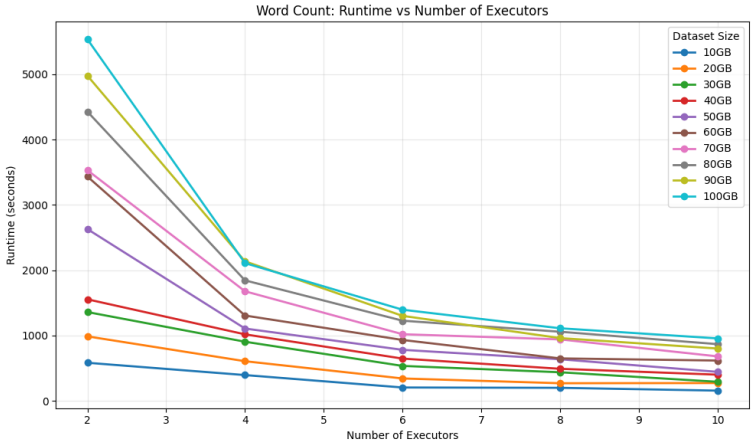
flattening of performance. At the time, the conclusion reached was partially due to the machine potentially being bottlenecked due to its architecture being made of a single machine being split into different virtual machines rather than a standard cluster being made of multiple machines used in conjunction and forming a network used concurrently to perform computation.



The performance scaling across the total cores can be split into three phases, in which the first rapid improvement is seen. From 4 cores to 12 cores the rate at which run time decreases is significant falling from around 9,000 seconds to 3,600 seconds. This suggests that the cleaning and tokenisation stages of the job are benefitting from the increased parallelism, allowing the tasks within each stage to be distributed across the worker nodes. Then as seen in previous performance tests the rate of improvement slows to a point that adding more cores provides diminishing returns, in which case those additional cores could be used to process another job.

5.3 Scalability Benchmarks

In order to evaluate the scalability of the Word Count implementation, the runtime was measured with the dataset size increasing along with the number of executors portrayed on a single graph to support the research of Amdahl's law as well as Gustafson's law which respectively are evident within the following graph.



The aspects of Amdahl's law present are through the gains in performance greatly decreasing as more executors are allocated due to there being limiting factors in place such as the I/O and the sequential aspects of the workload itself.

This experiment also demonstrates key aspects of Gustafson's law, which suggests that parallel systems are effective as users increase the problem size which is clearly visible due to the initial gap in performance being thinned down to a miniscule margin showing the efficiency gained by adding more resources.

Some minor fluctuations occurred as a result of the runtime data being gathered at different states of the cluster's physical architecture due to hardware issues occurring leading to increased maintenance work limiting access. This also cause failures when running extensive Spark jobs on the Cluster delaying the gathering of the relevant data needed.

5.4 Interim Recommendations

The data gathered from the 100 GB Wikipedia experiments in particular presents the optimisation that the Spark framework provides on the Big Data cluster. Based on the performance bottlenecks and results the following recommendations are to:

1. **Optimal Resource Allocation**

    - **Executor and Cores:** For both the applications, the largest performance increases came from 2 executors to 6 executors. Therefore, the aim should be to use 6 executors with 4 cores each for a workload of the size used within this project. Beyond this point, there are diminishing returns due to sequential part of the job and adding more executors outweighs any benefits of the added parallelism.
    - **Memory:** Increasing the executor memory over the 4 GB to 6 GB threshold gave negligible improvements and sometimes added to the runtime due to increased garbage collection overhead, meaning the pauses when clearing the unused memory became longer making the job last longer.
    - **Shuffle and partitions:** For the datasets used the default HDFS block size of around 128 MB was used, for the 100 GB dataset that meant splitting the workload into 800 partitions. This impacted the runtime in which this large amount of data could be processed, as having fewer partitions would mean the stress on each individual executor would be increased, slowing down the computation.

Throughout the process of utilising the Big Data cluster, the CIM help desk provided great insight into the approach to take when testing my code. They informed me of the architecture behind the universities cluster giving a further insight into the results gathered during the performance testing phase.

6 Software Engineering

6.1 Methodology

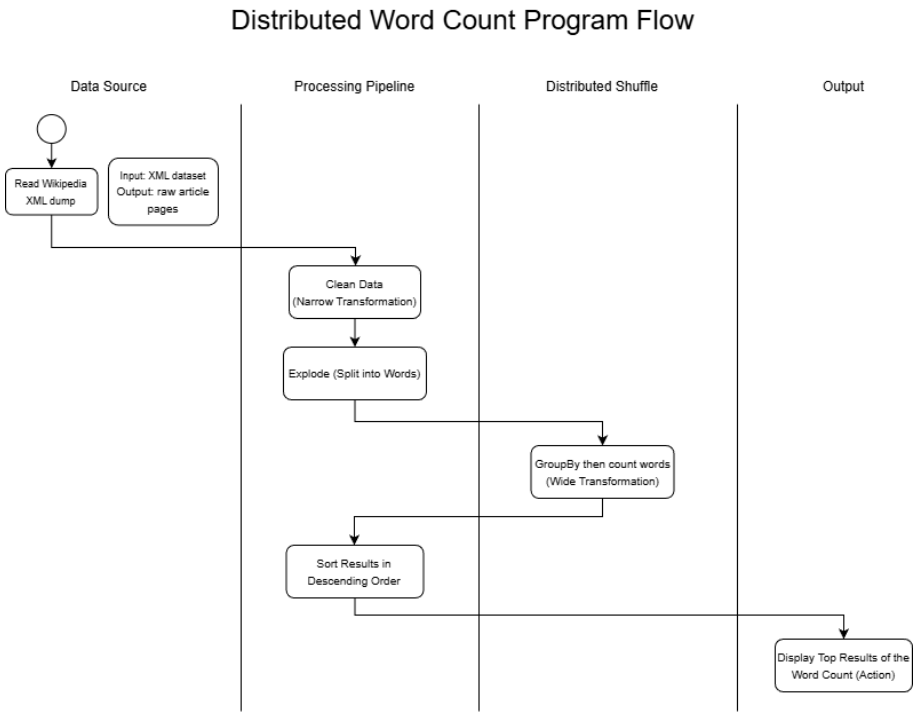
The project was initiated by gaining an understanding of the defined scope to be achieved in the early deliverables of the project description: implementing two Spark applications being the Word Count and Distributed Grep programs capable of processing a 100 GB Large Wikipedia Dataset.

An Iterative Development Approach was used in the project by developing and testing the Spark jobs on smaller, more manageable subsets of the larger dataset before being able to scale up. This approach of prototyping before a full scale implementation allowed me to debug any issues and gradually refine the output of the cleaned text and final outputs to validate the intended results. This saved a great deal of time as having to utilise the large dataset would be the antithesis of an optimal development strategy.

The GitLab repository was used as a version control system to manage the code and ensure segmentation between stages of development. Frequent, descriptive commits added to the evolution of the repository, along with issues which related the commits to an ultimate goal that was described showing the required actions to achieve the outcome and what the expected deliverables would be as the task set out by the issue was completed. Branches were used to partition the working code in main from the code in development for the Word Count program as an example to make sure the main codebase remained clean. A Dairy file tracking the regular progress was maintained to record any decisions, challenges and key concepts to focus on, as well as any supervisor meetings, through the course of building the project.

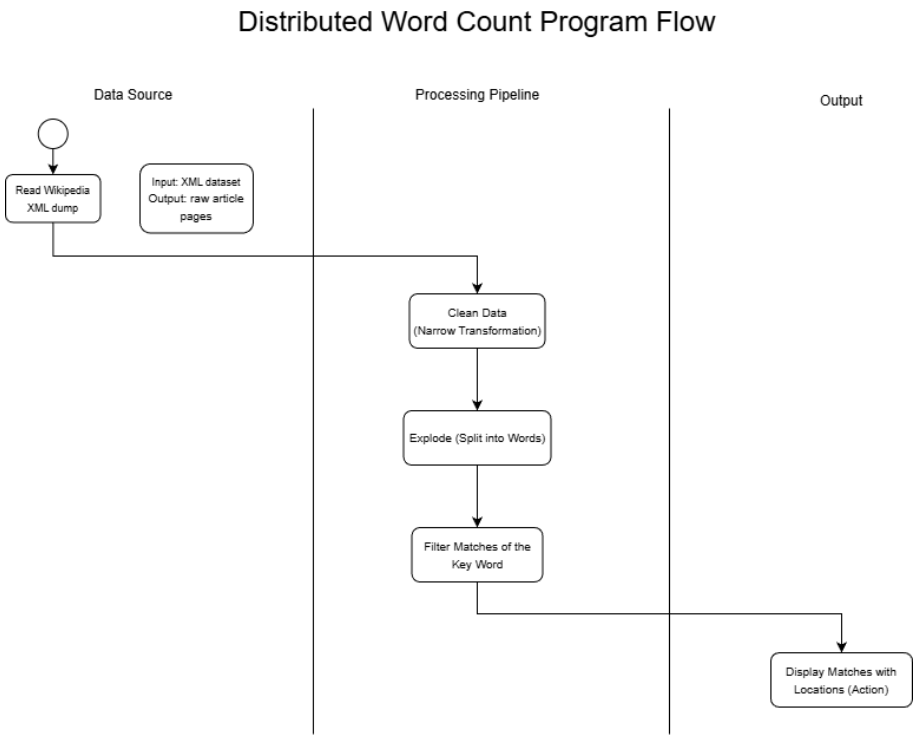
6.2 System Design

The Word Count program's flow is depicted by the following UML diagram:



This UML is a high level depiction of how the Word Count program's pipeline ran in a more linear view. This process in reality is split across the cluster and the partitions of the large dataset, with each worker node within the cluster contributing towards the overall computation.

The Distributed Grep program follows a similar design shown in the UML diagram below:



The flow is now split into three sections being the data source, meaning the HDFS storage system on the Big Data cluster, which then leads to the computations conducted on the data partitions and finally outputs some matches as a way to verify the program has completed the processing and displayed the results correctly.

References

[1] Bartley, K. (2025) 'Data Statistics (2025) - How much data is there in the world?', Rivery, 28 May. Available at: https://rivery.io/blog/big-data-statistics-how-much-data-is-there-in-the-world/ (Accessed: 27 November 2025).

[2] Jules S. Damji, Brooke Wenig, Tathagata Das, Denny Lee (2020) *Learning Spark: Lightning-Fast Data Analytics*, 2nd Edition, O'Reilly Media. (Accessed: 10 October 2025).

[3] Apache Hadoop: Advantages, Disadvantages, and Alternatives (2022) AltexSoft. Available at: https://www.altexsoft.com/blog/hadoop-pros-cons/ (Accessed: 5 October 2025).

[4] What is Big Data? — IBM (2024). Available at: `https://www.ibm.com/think/topics/big-data` (Accessed: 3 October 2025).

[5] Tang, N. (2014) 'Big Data Cleaning', in L. Chen et al. (eds) Web Technologies and Applications. Cham: Springer International Publishing, pp. 13–24. Available at: `https://doi.org/10.1007/978-3-319-11116-2_2` (Accessed: 20 November 2025).

[6] Dean, J. and Ghemawat, S. (2008) 'MapReduce: simplified data processing on large clusters', Communications of the ACM, 51(1), pp. 107–113. Available at: https://doi.org/10.1145/1327452.1327492 (Accessed: 21 November 2025).

[7] Benlachmi, Y. and Hasnaoui, M.L. (2020) 'Big data and Spark: Comparison with Hadoop', in 2020 Fourth World Conference on Smart Trends in Systems, Security and Sustainability (WorldS4). 2020 Fourth World Conference on Smart Trends in Systems, Security and Sustainability (WorldS4), pp. 811–817. Available at: https://doi.org/10.1109/WorldS450073.2020.9210353 (Accessed: 21 November 2025).

[8] Xiao, W. and Hu, J. (2020) 'A Survey of Parallel Clustering Algorithms Based on Spark', Scientific Programming, 2020(1), p. 8884926. Available at: `https://doi.org/10.1155/2020/8884926` (Accessed: 27 November 2025).

[9] Rajeh, W. (2022) 'Hadoop Distributed File System Security Challenges and Examination of Unauthorized Access Issue', Journal of Information Security, 13(2), pp. 23–42. Available at: https://doi.org/10.4236/jis.2022.132002 (Accessed: 25 October 2025).

[10] Zaharia, M. et al. (2010) 'Spark: Cluster Computing with Working Sets'. Available at: `https://www.usenix.org/legacy/event/hotcloud10/tech/full_papers/Zaharia.pdf` (Accessed: 29th October 2025).

[11] Manconi, A. et al. (2023) 'Framing Apache Spark in life sciences', Heliyon, 9(2), p. e13368. Available at: https://doi.org/10.1016/j.heliyon.2023.e13368. (Accessed: 15 November 2025).

[12] Zaharia, M. et al. (2012) 'Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing'. (Accessed: 27th November).

[13] Deep Dive into Spark SQL's Catalyst Optimizer (2015) Databricks. Available at: https://www.databricks.com/blog/2015/04/13/deep-dive-into-spark-sqls-catalyst-optimizer.html (Accessed: 2nd December 2025).

[14] 1. Introduction to Parallel Computing (2003). Available at: https://learning.oreilly.com/library/view/introduction-to-parallel/0201648652/ch01.html (Accessed: 13 November 2025).

[15] Hill, M.D. and Marty, M.R. (2008) 'Amdahl's Law in the Multicore Era', Computer, 41(7), pp. 33–38. Available at: https://doi.org/10.1109/MC.2008.209.

[16] Ahmed, N. et al. (2021) 'A parallelization model for performance characterization of Spark Big Data jobs on Hadoop clusters', Journal of Big Data, 8(1), p. 107. Available at: https://doi.org/10.1186/s40537-021-00499-7.